

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8
9 # Grouping by even and odd numbers and calculating the mean
10 grouped = series.groupby(series % 2 == 0).mean()
11
12 # Display the mean of even and odd numbers with labels
13 grouped.index = ['Even' if is_even else 'Odd' for is_even in
```

Average time

0.011 s

10.50 ms

Maximum time

0.026 s

26.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.

datafram...

```
18 name=input("New name: ")
19 age=int(input("New age: "))
20 new={'Name': name, 'Age': age}
21 df=df.append(new,ignore_index=True)
22
23 # Display the DataFrame after adding a new row
24 print("After adding a row:\n",df)
25
26 # Modifying a row
27 i=int(input("Index of row to modify: "))
28 nag=int(input("New age: "))
29 df.loc[i,'Age']=nag
30
```

Average time

0.091 s

91.50 ms

Maximum time

0.095 s

95.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

4.1.3. Student Information

01:55

Write a program to read a text file containing student information (name; age; and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

+

```
studentin...
5 data = pd.read_csv(file, sep=";", header=None, names=
6 ["Name", "Age", "Grade"])
7 print("First five rows:")
8
9 print(data.head(5))
10
11 # write your code here..
12
13 age=round(data['Age'].mean(),2)
14
15 print("Average age:", age)
16
17 print("Students with a grade up to B")
18
19 df=pd.DataFrame(data)
20
21 a=df[df['Grade']<='B']
22
23 print(a)
```

Terminal Test cases

< Prev Reset Submit Next >

4.2.1. Month with the Highest Total Sales

01:48

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,6,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

```
monthFor...
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 df['Date'] = pd.to_datetime(df['Date'])
9
10 df['Month'] = df['Date'].dt.to_period('M')
11
12 df['Total_Sales'] = df['Quantity'] * df['Price']
13
14 monthly_sales = df.groupby('Month')['Total_Sales'].sum()
15
16 # Find the month with the highest total sales
17
18 best_month = monthly_sales.idxmax()
19
```

Average time

0.026 s

25.67 ms

Maximum time

0.054 s

54.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Terminal Test cases

< Prev Reset Submit Next >

4.2.2. Best Selling Product

62.04

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 product_sales = df.groupby("Product")["Quantity"].sum()
9 best_product = product_sales.idxmax()
10 highest_quantity = product_sales.max()
11 # Display the result
12 print(f"Best selling product: {best_product}")
13 print(f"Total quantity sold: {highest_quantity}")
```

Average time

0.018 s

18.33 ms

Maximum time

0.038 s

38.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 38 ms

Debug

Terminal

Test cases

< Prev Reset Submit Next >

4.2.3. City that Sold the Most Products

61.51

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by city and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 city_sales = df.groupby("City")["Quantity"].sum()
11 best_city = city_sales.idxmax()
```

Average time

0.019 s

19.33 ms

Maximum time

0.040 s

40.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 40 ms

Debug

Expected output

Actual output

Terminal

Test cases

< Prev Reset Submit Next >

05:53

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date)
- Output the product pair/s that was sold most frequently

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Average time	Maximum time	Test Results
0.021 s 21.33 ms	0.044 s 44.00 ms	1 out of 1 shown test case(s) passed
		2 out of 2 hidden test case(s) passed

Terminal Test cases

[◀ Prev](#)
[Reset](#)
[Submit](#)
[Next ▶](#)

00:56 🔊 🌙 ✎ 🔗 —

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

[illegible]

Average time: 0.203 s
Maximum time: 0.203 s
1 out of 1 shown test case(s) passed

Terminal Test cases

◀ Prev Reset Submit Next ▶

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

00:15

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParSp	Ticket	Fare	Cabin	Embarked
-------------	----------	--------	------	-----	-----	-------	-------	--------	------	-------	----------

titanicDat...

```
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'],
16 drop_first=True)
17
18 # 4. Get the mean age of passengers
19 print(data['Age'].mean())
20
21
22 # 5. Get the median fare of passengers
23 print(data['Fare'].median())
24
25
```

Terminal Test cases

< Prev Reset Submit Next >

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

00:46

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParSp	Ticket	Fare	Cabin	Embarked
-------------	----------	--------	------	-----	-----	-------	-------	--------	------	-------	----------

titanicDat...

```
10 # 1. Calculate the survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Calculate the survival rate by embarked location
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 # 3. Calculate the survival rate by family size
17 print(data.groupby('FamilySize')['Survived'].mean())
18
19 # 4. Calculate the survival rate by being alone
20 print(data.groupby('IsAlone')['Survived'].mean())
21
22 # 5. Get the average fare by class
23 print(data.groupby('Pclass')['Fare'].mean())
24
```

Average time 0.097 s 97.00 ms Maximum time 0.097 s 97.00 ms 1 out of 1 shown test case(s) passed

Terminal Test cases

< Prev Reset Submit Next >

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

65/42

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex)
2. Get the number of non-survivors by gender (Sex)
3. Get the number of survivors by embarkation location (Embarked_S)
4. Get the number of non-survivors by embarkation location (Embarked_S)
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	ParCh	Ticket	Fare	Cabin	Embarked
-------------	----------	--------	------	-----	-----	-------	-------	--------	------	-------	----------

titanicDat...

```
8 # 1. Get the number of survivors by gender
9 print(data[data['Survived']==1]['Sex'].value_counts())
10
11 # 2. Get the number of non-survivors by gender
12 print(data[data['Survived']==0]['Sex'].value_counts())
13
14 # 3. Get the number of survivors by embarked location
15 print(data[data['Survived']==1]['Embarked_S'].value_counts())
16 # 4. Get the number of non-survivors by embarked location
17 print(data[data['Survived']==0]['Embarked_S'].value_counts())
18
19 # 5. Calculate the percentage of children (Age < 18) who
20 # survived
21 print(data[data['Age']<18]['Survived'].mean())
```

Average time

0.093 s

93.00 ms

Maximum time

0.093 s

93.00 ms

1 out of 1 shown test case(s) passed

Terminal

Test cases